OOPS PROJECT REPORT

Library Management System

Using Java and SQLite Database

Name: Ankita Sarkar

Course: B.Tech

Stream: CSE(IOT)

Enrolment Number: 12024052019021

Section: A

Class Roll Number: 25

Batch: 2024-28

1. Abstract

This report presents the design and implementation of a Library Management System (LMS) developed using Java, Object-Oriented Programming principles, and an SQLite database accessed via JDBC. The system provides core library operations including adding books, displaying available titles, issuing books to students, and processing returns. A simple graphical interface built with Java Swing allows users to view book records visually, while all data is stored persistently in a local SQLite database file (library.db). The project demonstrates practical application of OOP concepts such as encapsulation, abstraction, inheritance, and polymorphism within a layered software architecture organised into packages.

2. Introduction

Managing library resources manually is time-consuming and prone to errors. A software-based Library Management System streamlines these tasks by maintaining accurate, persistent records of books and students, and automating processes such as issuing and returning books.

This project implements a console and GUI-based LMS using the Java programming language. Java was chosen for its platform independence, strong OOP support, and rich standard libraries. SQLite, a lightweight file-based relational database, provides persistent storage without requiring a separate database server. The JDBC (Java Database Connectivity) API bridges the application and the database. Java Swing is used to render a simple graphical view of the book inventory.

The system is structured into well-defined packages — database, models, services, and ui — reflecting a layered architecture that separates concerns and promotes maintainability.

3. Objectives

- To design a working Library Management System using core Java and SQLite.
- To apply OOP principles including encapsulation, abstraction, inheritance, polymorphism, and exception handling.
- To implement persistent data storage using JDBC and SQLite.
- To build a basic graphical interface using Java Swing for displaying book records.
- To structure the project into logical packages for clarity and maintainability.
- To demonstrate console-based user interaction for library operations.

4. Problem Statement

Traditional library management using paper registers is inefficient and error-prone. Records of issued and returned books are difficult to track, and manual search through large inventories wastes time. There is also no reliable mechanism to check whether a book is currently available or issued.

This project addresses these problems by developing a software system that can add and store book records, track the availability status of each book, associate issued books with student records, and display up-to-date book information through both a console interface and a simple GUI.

5. System Analysis

5.1 Existing System

In a traditional manual library system:

- Books and student details are recorded in physical registers.
- Locating a specific book or checking its availability requires manual searching.
- Issue and return records are maintained in separate ledgers.
- There is no automated notification or status tracking for overdue or issued books.
- Data is vulnerable to loss through damage or misplacement.

5.2 Proposed System

The proposed system replaces manual processes with a Java-based application backed by an SQLite database. Key improvements include:

- Digital storage of book and student records in a structured relational database.
- Instant lookup and status checks for any book (available or issued).
- Console-based menu for performing all core operations.
- A Swing GUI panel for viewing the complete book list in a tabular format.
- Persistent storage ensuring data is retained across application sessions.
- Modular codebase making future enhancements straightforward.

6. System Design

6.1 Architecture Description

The system follows a three-layer architecture:

Layer	Package	Responsibility
Presentation Layer	ui	Java Swing GUI — displays book list in a table
Business Logic Layer	services	LibraryService — handles add, issue, return operations
Data Access Layer	database	DBConnection, DBSetup — manages SQLite connection and schema
Model Layer	models	Book, Student — data entities with getters

The Main class acts as the entry point, initialising the database, launching the Swing UI, and starting the console menu loop.

6.2 Module Description

database.DBConnection: Establishes and returns a JDBC connection to the library.db SQLite file. Ensures a single point of database access.

database.DBSetup: Creates the books and students tables on first run if they do not already exist, using SQL CREATE TABLE IF NOT EXISTS statements.

models.Book: Represents a book entity with private fields (id, title, author, isIssued) and public getter methods. Demonstrates encapsulation.

models.Student: Represents a student with id and name fields and corresponding getters.

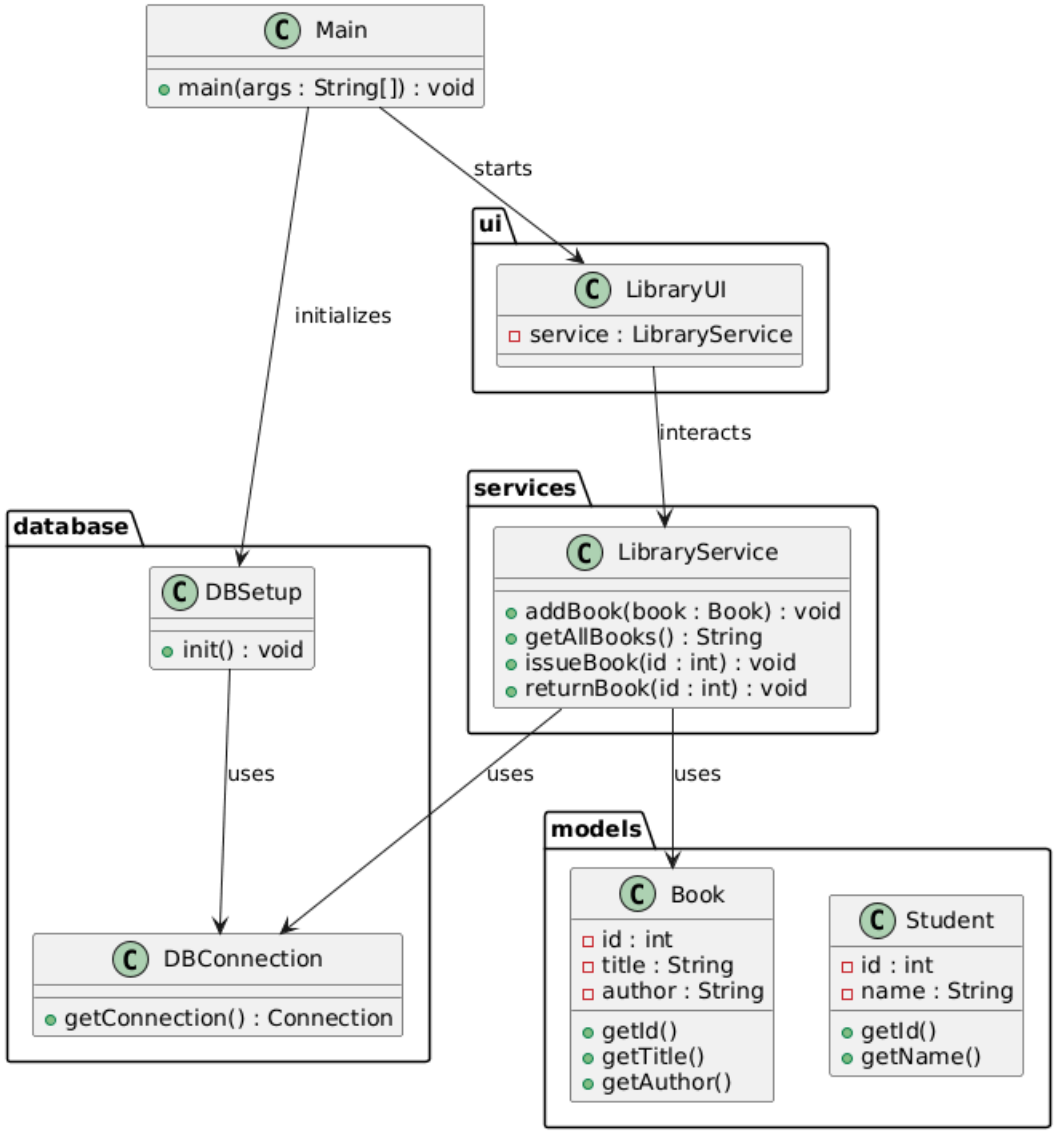
services.LibraryService: Core business logic class. Contains methods for addBook(), getAllBooks(), issueBook(), and returnBook(). Interacts directly with the database via JDBC PreparedStatements.

ui.LibraryUI: A JFrame-based window displaying all books in a JTable. Retrieves data from LibraryService and presents it in a scrollable table view.

Main: Initialises DBSetup, creates a LibraryService instance, launches the Swing UI on the Event Dispatch Thread, and runs the console-based interaction loop.

7. UML Class Diagram

The class diagram below illustrates the relationships between the main components of the system. Arrows indicate usage/dependency relationships.



8. Database Design

The system uses an SQLite database file named library.db with two tables.

8.1 Table: books

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique book identifier
title	TEXT	NOT NULL	Title of the book
author	TEXT	NOT NULL	Author of the book
isIssued	INTEGER	DEFAULT 0	0 = Available, 1 = Issued

8.2 Table: students

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique student identifier
name	TEXT	NOT NULL	Full name of the student

The isIssued field in the books table uses an integer flag (0 or 1) rather than a boolean, as SQLite does not have a native boolean type. The LibraryService layer maps this to a Java boolean when constructing Book model objects. The two tables are currently independent; a future version could add a foreign key linking an issued book to a student record.

9. Implementation Details

9.1 Technologies Used

Technology	Version / Type	Purpose
Java SE	JDK 11+	Core programming language
JDBC	java.sql package	Database connectivity API
SQLite	File-based RDBMS	Persistent local storage
SQLite JDBC Driver	sqlite-jdbc JAR	Bridge between Java and SQLite
Java Swing	javax.swing package	Graphical user interface
IDE	IntelliJ IDEA / Eclipse	Development environment

9.2 OOP Concepts Applied

Encapsulation:
Encapsulation is achieved by declaring all the data members in the Book and Student classes as private. Direct access to these fields is restricted, and they can only be accessed or modified through public getter methods. This ensures data hiding and protects the internal state of objects from unintended interference. For example, attributes such as id, title, and author in the Book class cannot be modified directly from outside the class, which helps maintain data integrity. Encapsulation also improves code maintainability, as any future changes to data representation can be handled within the class without affecting other parts of the system.

Abstraction:
Abstraction is implemented by separating the system into different layers, where only essential functionalities are exposed to the user. The LibraryService class abstracts all database-related operations such as adding, retrieving, issuing, and returning books. The user interface (LibraryUI) and the main application (Main) do not directly interact with SQL queries or database connections. Instead, they invoke high-level methods like addBook(), issueBook(), and getAllBooks(). This hides the complexity of database operations from the rest of the system and makes the code easier to understand, modify, and extend.

Inheritance:
Inheritance is applied conceptually in the design of model classes, allowing future extensibility. The Book and Student classes can act as base classes for more specialized entities. For instance, a DigitalBook class can inherit from Book and include additional attributes such as file format or download link, while a ResearchStudent or PostGraduateStudent class can extend the Student class with extra academic details. This promotes code reuse and avoids duplication by allowing common properties and methods to be inherited from parent classes. It also makes the system scalable for future enhancements without modifying the existing codebase.

Polymorphism:
Polymorphism allows objects to be treated as instances of their parent class while enabling different behaviors. In this project, polymorphism can be applied through method overriding, where subclasses of Book or Student override methods such as toString() or display-related methods to provide customized output. Java's runtime polymorphism ensures that the correct method implementation is executed based on the object type at runtime. This makes the system flexible and adaptable, allowing new features to be added without changing existing logic in the service layer.

Exception Handling:
Exception handling is used extensively to ensure robustness and reliability of the system. All database operations are enclosed within try-catch blocks to handle exceptions such as SQLException, invalid inputs, or connection failures. For example, if a database connection cannot be established or a query fails, the system catches the exception and displays an appropriate error message instead of crashing. This improves user experience and ensures smooth execution. Additionally, input validation in the UI layer prevents invalid data (such as non-numeric IDs) from being processed, further enhancing system stability.

10. Results and Output

Upon launching the application, the Main class initialises the database and ensures that the required tables exist. The Swing window (LibraryUI) opens and displays all currently stored books in a table with columns for ID, Title, Author, and Availability Status.

The console menu simultaneously presents the following options to the user:

1. Add Book — prompts for title and author, then inserts the record into the books table.
2. Display Books — lists all books from the database in the console.
3. Issue Book — prompts for a book ID and student ID. If the book is available, sets isIssued to 1.
4. Return Book — prompts for a book ID. If the book is currently issued, resets isIssued to 0.
5. Exit — terminates the application.

All changes made through the console are immediately reflected in the SQLite database. The Swing GUI can be refreshed or reloaded to show updated data. The system correctly prevents issuing a book that is already issued and handles invalid inputs gracefully through exception handling.

Operation	Input	Expected Output
Add Book	Title: Java Basics, Author: James	Book added successfully
Display Books	(none)	List of all books with status
Issue Book	Book ID: 1, Student ID: 2	Book issued successfully
Return Book	Book ID: 1	Book returned successfully
Issue (already issued)	Book ID: 1	Error: Book already issued

11. Advantages

- All data is stored persistently in library.db, eliminating the need for manual record-keeping.
- The layered package structure makes the codebase easy to read, test, and extend.
- SQLite requires no installation or server configuration, making deployment straightforward.
- The Swing GUI provides a visual interface accessible to non-technical users.
- Exception handling ensures that runtime errors do not crash the application.
- The system can be run on any platform with a Java Runtime Environment installed.

12. Limitations

- There is no user authentication; any user who runs the application has full access.

- The system does not track which student has issued which book in the current implementation.
- There is no due date or fine management for overdue books.
- The Swing GUI does not auto-refresh when data changes through the console; it must be manually reloaded.
- SQLite is not suitable for multi-user concurrent access in a networked environment.
- There is no search or filter functionality for the book list.

13. Future Enhancements

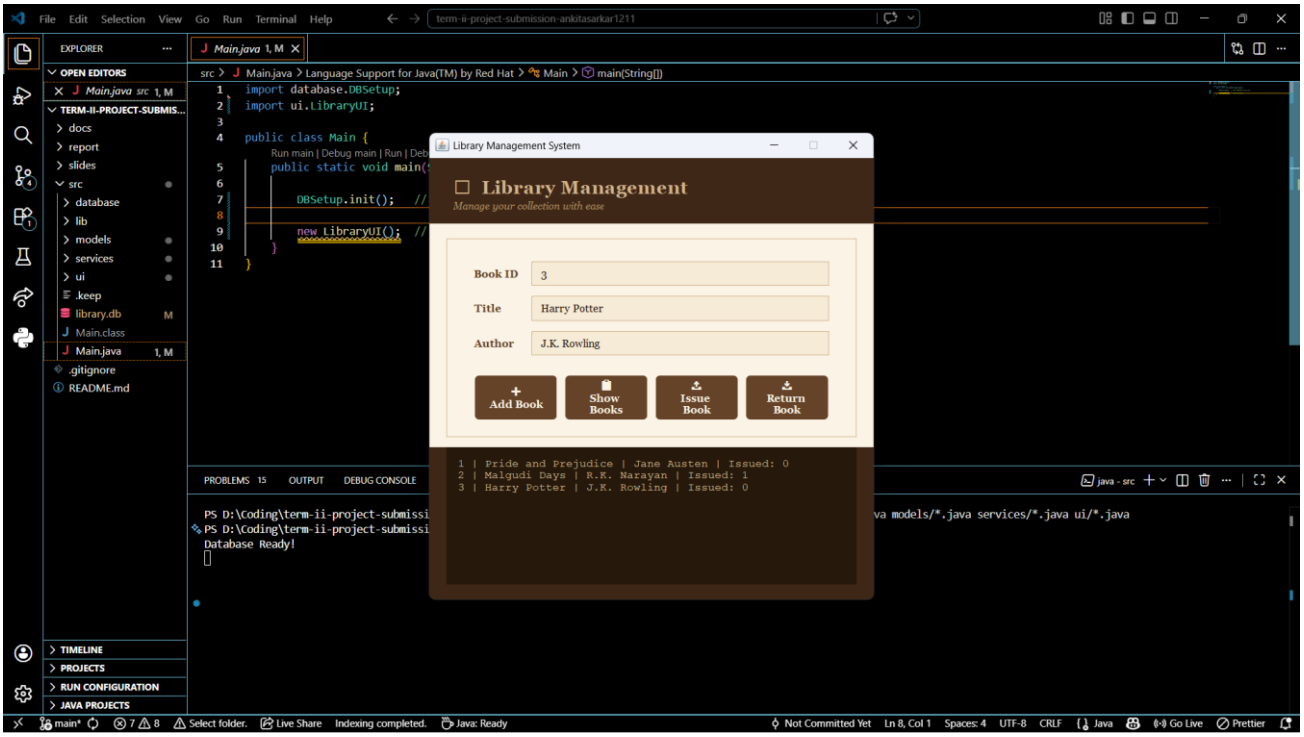
- Add a student management module to register students and maintain issue history.
- Implement a due date system with automatic fine calculation for late returns.
- Introduce a search and filter feature to find books by title or author.
- Add a login system with role-based access (Librarian vs. Student).
- Migrate to a MySQL or PostgreSQL database for multi-user support.
- Build a web-based interface using Java Servlets or a REST API framework.
- Enable auto-refresh of the Swing GUI using event listeners or a background thread.

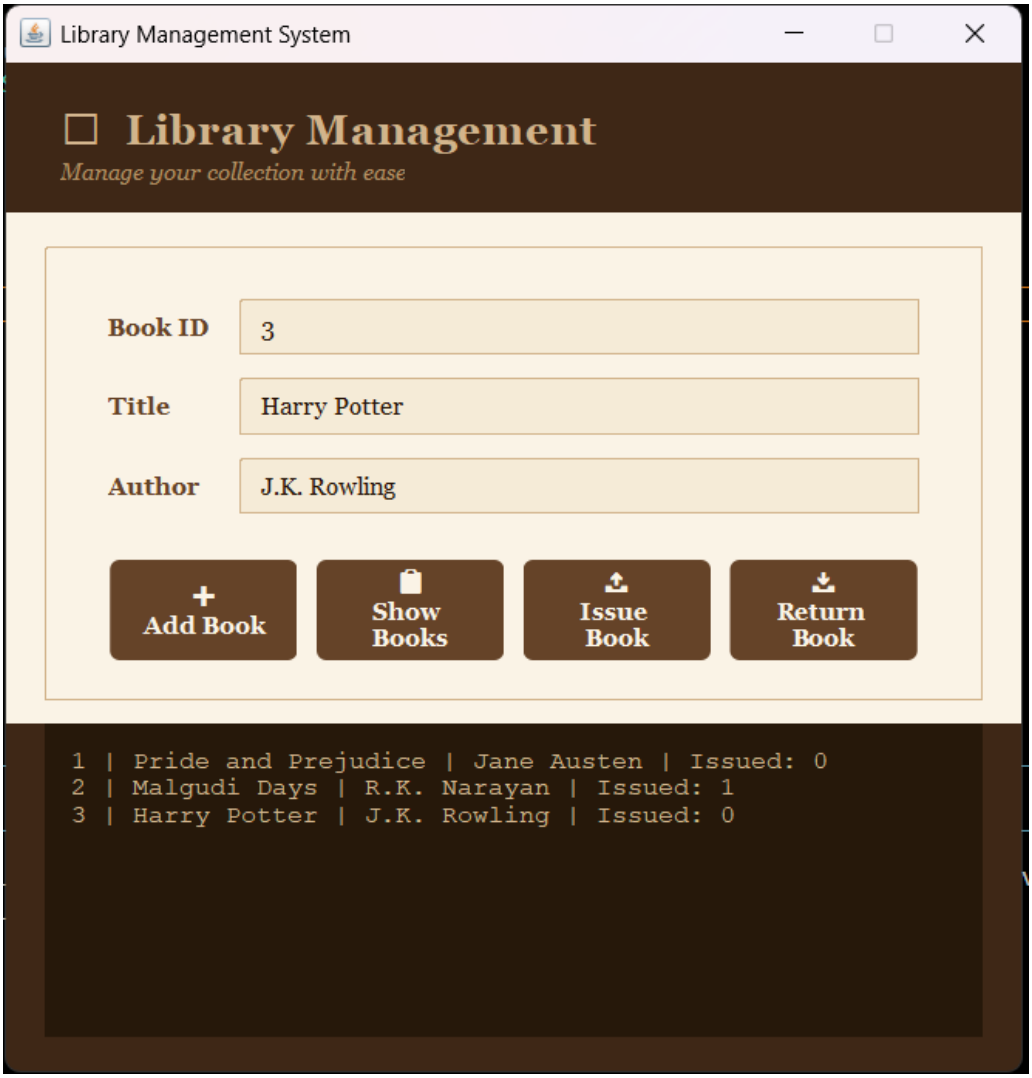
14. Conclusion

This project successfully demonstrates the development of a functional Library Management System using Java, JDBC, SQLite, and Java Swing. The system covers essential library operations — adding books, issuing, and returning — with all data stored persistently in a local database.

The project reinforces core OOP principles through practical implementation. Encapsulation ensures data safety in model classes; abstraction separates database logic from the UI; and exception handling provides robustness during runtime. The structured use of packages reflects good software design practices appropriate for a second-year CSE project.

While the current version is a console and basic GUI application, the architecture is extensible and can be enhanced with authentication, fine management, and network-based multi-user support in future iterations. Overall, this project provides a solid foundation for understanding database-driven application development in Java.


Library Management System
Library Management
Manage your collection with ease
Book ID
3
Title
Harry Potter
Author
J.K. Rowling
Add Book
Show Books
Issue Book
Return Book
1 | Pride and Prejudice | Jane Austen | Issued: 0
2 | Malgudi Days | R.K. Narayan | Issued: 1
3 | Harry Potter | J.K. Rowling | Issued: 0


Library Management System
Library Management
Manage your collection with ease
Book ID
3
Title
Harry Potter
Author
J.K. Rowling
Add Book
Show Books
Issue Book
Return Book
1 | Pride and Prejudice | Jane Austen | Issued: 0
2 | Malgudi Days | R.K. Narayan | Issued: 1
3 | Harry Potter | J.K. Rowling | Issued: 0